

Integrating rush orders into existent schedules for a complex job shop problem

Patrick Moratori · Sanja Petrovic ·
José Antonio Vázquez-Rodríguez

Published online: 3 March 2010
© Springer Science+Business Media, LLC 2010

Abstract This paper investigates the problem of inserting new rush orders into a current schedule of a real world job shop floor. Effective rescheduling methods must achieve reasonable levels of performance, measured according to a certain cost function, while preserving the stability of the shop floor, i.e. introducing as few changes as possible to the current schedule. This paper proposes new and effective match-up strategies which modify only a part of the schedule in order to accommodate the arriving jobs. The proposed strategies are compared with other rescheduling methods such as “right shift” and “insertion in the end”, which are optimal with respect to stability but poor with respect to performance, and with “total rescheduling” which is optimal with respect to performance but poor with respect to stability. Our results and statistical analysis reveal that the match-up strategies are comparable to the “right shift” and “insertion in the end” with respect to stability and as good as “total rescheduling” with respect to performance.

1 Introduction

Traditionally, research on scheduling has been mostly concerned with static and deterministic environments in which problem data is deterministic, known in advance and the processing of operations occurs under fully controlled conditions [1]. Real world production shops, however, are unpredictable and dynamic: machines breakdown [2–4], workers get sick [5], operations take more or less time than

planned [6], suppliers fail to deliver on time [7, 8], new production orders arrive to the shop floor [9–11], clients cancel orders [12, 13], etc. In this paper, we are concerned with one particular type of disruption that is common to most shop floors; namely, the arrival of new rush orders that have to be integrated into an existent schedule. The insertion of new jobs into an existent schedule, as well as most of the other types of disruptions, may require the total or partial rescheduling of previously allocated and new jobs [14]. Rescheduling algorithms are desired to produce schedules with similar levels of *Performance* as the initial schedule, measured according to a certain cost function. Moreover, it is also desirable that new schedules maintain the *Stability* of the system by introducing as few changes as possible to the original schedule.

Approaches which deal with disturbances in the production shop can be classified into three main groups [15]: (1) reactive scheduling algorithms, which select, in an online fashion, the operation to be processed next on available machines [12, 16, 17]; (2) robust scheduling algorithms, which anticipate disturbances and create a schedule that can absorb, at least partly, the effect of disruptions that may occur on the shop floor [18–21]; and (3) predictive-reactive scheduling algorithms, in which a schedule which optimises shop floor *Performance* is initially constructed and it is modified when a disruption occurs [22–24]. This paper investigates algorithms that belong to the class of predictive-reactive approaches, so called match-up algorithms [25]. These algorithms aim to maintain the *Stability* and *Performance* of a schedule by modifying only a part of the initial schedule, referred to as rescheduling horizon, when a disturbance occurs. Their motivation is that once having an initial optimum schedule the best is to return to such optimum schedule as quickly as possible after a disruption occurs. In other words, the idea is to “match-up” the disturbed sched-

P. Moratori (✉) · S. Petrovic · J.A. Vázquez-Rodríguez
School of Computer Science, University of Nottingham,
Nottingham, NG8 1BB, UK
e-mail: pbm@cs.nott.ac.uk

ule to the initial one, as quickly as possible. Match-up strategies achieve this by only modifying the schedule within the rescheduling time window, keeping unchanged the schedule before and after the rescheduling window.

Match-up strategies have been only used for relatively simple scheduling problems. For instance, [25–27] consider single machine problems, [28] considers a single stage with parallel machines problem and in [29] a match-up strategy coupled with a branch and bound algorithm is used for a flow shop problem. Knowledge-based systems are used for the job shop problem in [5, 7, 8, 30]. These systems resemble match-up approaches in that they also consider only a part of the schedule for rescheduling. At every decision point, these methods consider the most constrained part of the schedule. Then, they select, based on some stored knowledge, an appropriate rescheduling strategy. A drawback of these systems is that a single conflict (constraint violation) may propagate to the remaining part of the schedule, requiring several repairs and compromising the *Stability* of the schedule. Another job shop scheduling problem but with machine breakdown was investigated in [31]. In this study, the developed heuristic collects affected jobs and shifts them to the right of the schedule, as required. In [32], a genetic algorithm considers both, the *Stability* and the *Performance* measures in order to generate optimal solutions for single machine problems.

In this paper, we investigate a complex production job shop problem in Sherwood Press, a printing company based in Nottingham, UK. A static version of this scheduling problem was investigated by [33]. The scheduling of printing orders can be defined as a job shop problem with parallel machines, machine eligibility and sequence dependent setup times. In this paper, the scheduling problem is formulated as dynamic in the sense that new printing orders arrive every day and have to be integrated in the existing schedule. When a disruption occurs, i.e. a rush order arrives to the system, the first and crucial step in match-up algorithms is to define the rescheduling horizon. In this paper, 8 different strategies for calculating the rescheduling horizon, each leading to a different match-up algorithm, are proposed and compared. Our strategies are compared with other rescheduling methods such as “right shift”, “insertion in the end”, which are optimal with respect to *Stability* but of poor *Performance* and “total rescheduling” which is optimal with respect to *Performance* but of poor *Stability*. The 8 proposed match-up algorithms and the other rescheduling strategies were tested on multiple instances obtained from Sherwood Press. Our results and statistical analysis reveal that the match-up strategies are comparable to the “right shift” and “insertion in the end” strategies with respect to *Stability* and are as good as “total rescheduling” with respect to *Performance*.

The rest of the paper is organised as follows. Section 2 introduces a real world job shop rescheduling problem present

at Sherwood Press and defines *Performance* and *Stability* in a formal way. Section 3 describes the match-up algorithm in detail and proposes 8 strategies for calculating the rescheduling horizon. Section 4 describes the experimental methodology and presents, analyses and discusses the obtained results. Section 5 concludes the paper and describes our present and future work.

2 Problem statement

The job shop scheduling problem in Sherwood Press requires the allocation of a number of jobs onto 18 machines grouped into 7 working centres: for printing, cutting, folding, card-inserting, embossing/debossing, gathering, and finishing (stitching/trimming/packing). Some machines in each work centre are identical and are treated as parallel machines. Each job is subject to precedence constraints, i.e. it has a predefined sequence of machines on which it has to be processed. Each job is assigned a release and a due date which denote the earliest time when the job can start its processing and the time when the job is desired to be completed, respectively. The processing of a job on a machine is referred to as an operation and each operation requires a certain processing time. Each job is also assigned to a category which indicates how strict is its due date: jobs in the first category must be completed by their due date or earlier, jobs in category 2 are given a two days tolerance period and jobs in category 3 are given up to one week of tolerance. Note that any failure to finish strictly on time is always penalised. The magnitude of the penalty, however, depends on the category of the job and the severity of the delay. Jobs also require a certain setup time in the printing process. This time depends on the sequence of the operations and is larger if jobs requiring different colours are processed one after the other.

Every day, new jobs arrive in the shop floor. The current policy of the company is to keep the existing schedule unchanged for 2 days (48 hours). After that time the new job can start its processing. In this research, we consider new jobs that are rush orders. The task is to accommodate the new job into the existing schedule, as soon as possible, in such a way that the *Performance* and the *Stability* of the schedule, defined next, are maximised.

2.1 Performance

Five criteria were considered in the evaluation of *Performance* of schedules: (1) the average weighted tardiness of jobs, (2) the number of tardy jobs, (3) the total setup time, (4) the total idle time of machines, and (5) the total flow time (see [33] for a detailed description of these objective functions). The values of these criteria are mapped into satisfaction levels and aggregated into a final *Performance* measure. The higher the *Performance*, the higher the satisfaction

with the produced schedule and, therefore, it has to be maximised.

2.2 Stability

The *Stability* measure introduced here is an adaption of the one proposed in [31], which we modified in order to cope with the parallel machines environment. Let M be the number of machines on the shop floor and let O_i be the number of operations $j = 1, \dots, O_i$ that have to be processed on machine $i = 1, \dots, M$. The following measure of sequence similarity $Reward_i \in [0, 1]$ is assigned to each machine i :

$$Reward_i = \sum_{j=1}^{O_i-1} \frac{Reward_{ij}}{O_i - 1}, \quad (1)$$

where

$$Reward_{ij} = \begin{cases} 1 & \text{if operation } j+1 \text{ remains successor} \\ & \text{of operation } j \text{ on machine } i; \\ 0 & \text{otherwise.} \end{cases}$$

In order to keep the sequence of operations on each machine as unchanged as possible, the sum of rewards of the machines has to be maximised. Since we are dealing with parallel machines, we consider the following special cases for the calculation of $Reward_i$:

1. If machine i is empty and stays empty after rescheduling, then $Reward_i = 1$.
2. If machine i has originally only one operation and in the new schedule any number N , then $Reward_i = 1$.
3. If machine i is empty and any number N of operations are assigned to it in the new schedule, then $Reward_i = 0$.
4. If machine i has N operations and becomes empty in the new schedule, then $Reward_i = 0$.

The rationale of these rules is to penalise drastic changes in machine workload. In case 1, the rescheduling does not affect the workload of the machine, which is highly rewarded. In case 2, the sequence of operations is not affected by adding any number of operations. In cases 3 and 4, the workload is completely changed, which is penalised by setting $Reward_i = 0$. The *Stability* measure, to be maximised, is defined as follows:

$$Stability = \frac{1}{M} \sum_{i=1}^M Reward_i. \quad (2)$$

3 Match-up strategies for rescheduling

This section introduces a match-up algorithm for dynamic scheduling of rush orders. The pseudocode of the match-up

Input: An initial schedule S , a new job j , *initialStart*

Output: A new schedule with job j integrated

1. Let *startPoint* be the latest completion time among operations whose processing time is crossed by *initialStart*
2. Calculate *endPoint* by collecting idle time on the machines required by the new job
3. Let O be the set of operations within *startPoint* and *endPoint* plus the operations of job j
4. Update the release and due dates of jobs in O so that they lie within *startPoint* and *endPoint*
5. Let operations in O define a new scheduling problem S'
6. Solve S' using the genetic algorithm from [33]
7. Integrate S' into S , checking and removing overlaps

Fig. 1 Pseudo-code of the match-up algorithm

algorithm proposed is given in Fig. 1. The match-up algorithm has three phases which are firstly outlined followed by a detailed description. In the first phase, steps 1–2, the rescheduling horizon, within which the operations of the new job will be accommodated, is defined. In the second phase, steps 3–6, a new scheduling problem containing operations within the calculated horizon and the new job are defined and solved. In the third phase, step 7, the newly generated schedule is integrated into the original one, checking and repairing possible overlaps between the unchanged part of the schedule and the newly generated partial schedule.

In step 1 of the algorithm, *initialStart* denotes the time of the arrival of the new job j increased by 48 hours, as required by the company. For example, Fig. 2(a) shows, using a Gantt chart, the processing requirements and precedence constraints of a new job. This needs to be processed either on machine M1 or machine M2, which are parallel machines, on machine M3 and on machine M4. The operations of the initial schedule that have started their processing before *initialStart* must be completed before the rescheduling process begins. Therefore, all the operations that are being processed at *initialStart* are collected. These are the operations in Fig. 2(b) that cross the *initialStart* line, i.e. jobs 1, 3 and 4 on machines M1, M3 and M4, respectively. The operation with the highest completion time among the collected operations determines the *startPoint*. In Fig. 2(c), this is job 3 on M3. All the operations that can be completed before *startPoint* will resume their processing (job 2 on M2).

In order to calculate *endPoint* (step 2), the algorithm collects idle times on the machines that are required by the new job. Four different strategies Q1–Q4, illustrated with an example in Fig. 3, are introduced. Suppose that a job consisting of two operations arrives in the shop floor. The new job requires 4 and 2 processing time units on machines MA

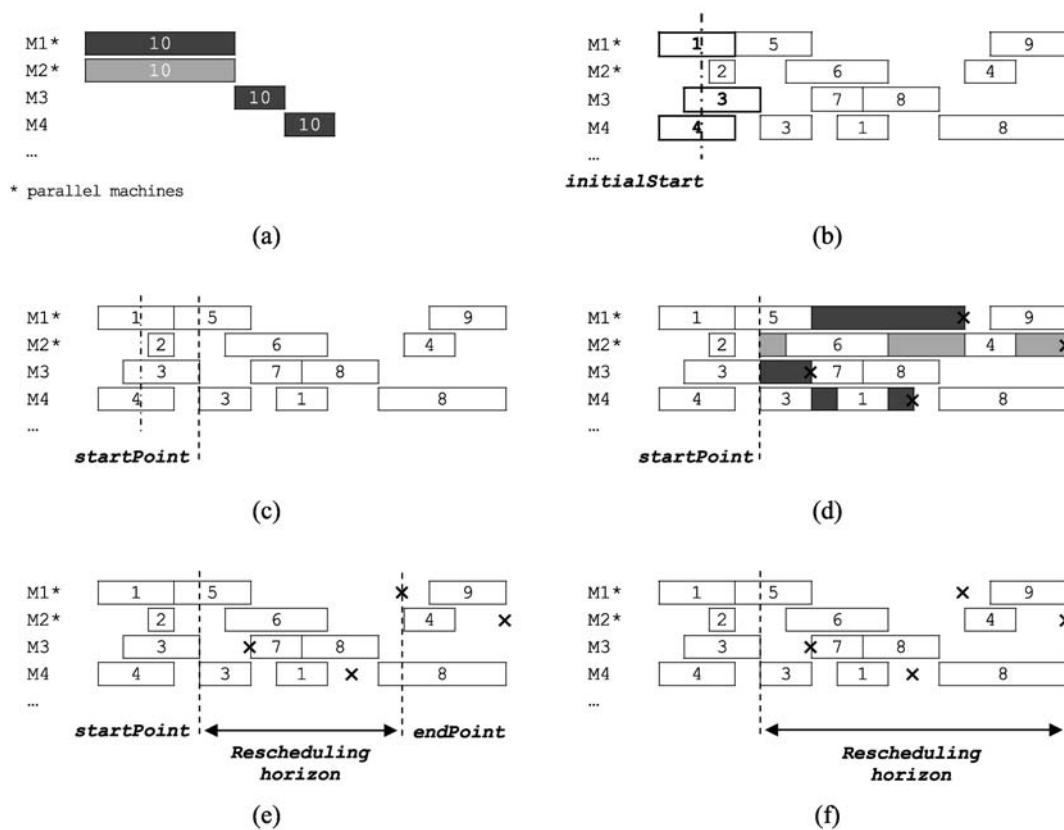


Fig. 2 Example of the calculation of the rescheduling horizon; (a) the new job processing requirements, (b) the calculation of *initialStart*, (c) the *startPoint* definition, (d) the collection of idle time on machines, (e) the calculation of *endPoint* using strategy S1 and (f) the calculation of *endPoint* using strategy S5

and MB, respectively, see Fig. 3(a). Four different strategies are introduced to collect idle times on machines:

- Q1 Collect idle time on the required machines accumulating enough time, not necessarily as a single time window, until the collected idle time equals the new job processing requirements, see Fig. 3(b).
- Q2 Collect idle time contained in a single time window, as in Fig. 3(c).
- Q3 As Q1, but considering the precedence constraints imposed on the operations of the new job. Fig. 3(d) shows idle time being collected on machines MA and MB. Note that the collection of time on machine MB can start only after enough time has been collected on machine MA.
- Q4 As Q2, but considering the precedence constraints of the new job, as demonstrated in Fig. 3(e).

Let C_i be the completion time of a new job on machine i ; then *endPoint* is defined as the maximum of C_i , $i = 1, \dots, M$. Note that in the case of parallel machines, there are two or more possible completion time values for the same operation. Just one of these is considered for the calculation of *endPoint*. Here we use either the earliest or

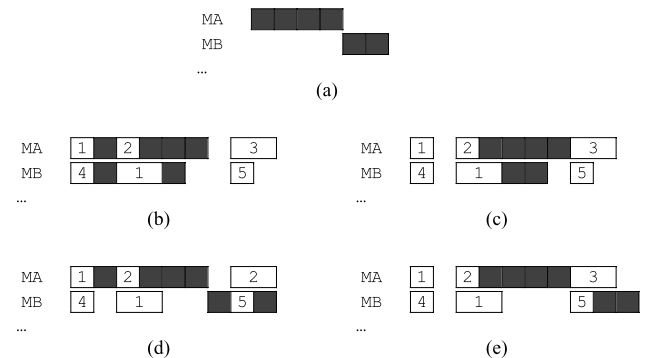


Fig. 3 Example of the collection of idle times; (a) the new job requirements, (b) idle time collection using strategy Q1, (c) Q2, (d) Q3 and (e) Q4

the latest of the completion times. In this way, we define 8 strategies: S1–S4 that collect idle time using Q1–Q4, respectively, and consider the earliest completion time among the parallel machines, and S5–S8 which collect idle times using Q1–Q4, respectively, but consider the latest completion time. Figure 2(d) demonstrates the idle time collection using strategy S1. Since the two accumulated times that extend the

latest are on machines M1 and M2, which are parallel machines, there are two options for the definition of *endPoint*. The first option, using S1, is to consider the earliest completion time, as in Fig. 2(e). The other option, using S5, is to consider the latest completion time, as in Fig. 2(f).

Once the rescheduling horizon has been calculated, a new scheduling problem is defined. This problem requires the scheduling of the operations that lie within the rescheduling horizon and the operations of the new job. Let O be the set of all these operations (step 3) and let r_o and d_o be the release time and the due date of operation $o \in O$, respectively, in the original schedule. In order to keep the operations in O within the rescheduling horizon after rescheduling, the release times of the operations in O are set to $r_o^{new} = \max\{r_o, startPoint\}$, $\forall o \in O$, and the due dates are set to $d_o^{new} = \min\{d_o, endPoint\}$, $\forall o \in O$, (step 4). In step 5, a new scheduling problem that considers the operations in O with their updated release and due dates and with the objective of maximising the *Performance* measure is defined. This problem is solved in step 6 using the same fuzzy multi-criteria genetic algorithm that was used to construct the initial schedule [33].

In the final phase (step 7), the initial partial schedule contained within the rescheduling horizon is replaced by the new schedule generated in step 6. It may be the case that the completion times of one or more operations in the new schedule are out of the rescheduling horizon. If this is the case, there is the possibility that such operations overlap with the operations from the initial schedule that start after the *endPoint*. These cases are identified, and operations from the initial schedule are shifted to the right as necessary in order to restore feasibility.

The GA used to obtain the initial schedule and to do the rescheduling in step 6 of the proposed algorithm is the one proposed in [33] which was designed and tuned for the scheduling problem of Sherwood Press. The GA represents solutions as sequences of dispatching rules that determine the order of processing of operations on each of the machines. At initialisation, a randomly selected dispatching rule is assigned to each machine. Crossover is done in a uniform fashion and mutation is performed by swapping pairs of elements in the solution. The algorithm is elitist, meaning that the best solution is kept throughout the run of the algorithm. The population size and number of generations were set to 50 and 200, respectively, whereas the crossover and mutation rate were set to 70% and 20% respectively.

4 Experiments on real world data

Data sets obtained from Sherwood Press are used to test the *Performance* and *Stability* of the developed match-up algorithm. The data of the production orders of several months

were used to produce schedules. The saturation, *sat*, is used as an indicator of the amount of idle time in the shop floor and is calculated as the ratio between the makespan of the initial schedule and the sum of processing times of all operations:

$$sat = \frac{C_{max}}{\sum_{i=1}^n \sum_{j=1}^M p_{ij}} \quad (3)$$

where C_{max} is the length of the initial schedule (makespan) and p_{ij} is the processing time required by job i , $i = 1, \dots, n$ on machine j , $j = 1, \dots, M$. A large *sat* value indicates a highly saturated schedule, i.e. with a small amount of idle time. Three months with different saturation levels were selected and used to generate three instances. The saturation levels for each considered month are 0.272, 0.422 and 0.541. These values correspond to low, medium and high (i.e. *sat* $\in$ {low, medium, high}) saturation levels, respectively.

In each instance, new arriving jobs were randomly generated taking into account two parameters. The first parameter is the time of insertion of the new job, *insTime* $\in$ {beginning, middle, end} where “beginning”, “middle” and “end” refer to a *startPoint* equal to 10%, 50% and 80% of the makespan of the initial schedule, respectively. The reason for considering the *insTime* follows the observation that the workload of the shop varies at different points in the schedule. The workload at the beginning and in the middle of the schedule, for instance, is often higher than the workload in the middle of the schedule, which is higher than the workload at the end of the schedule. The second parameter is the number of operations in the new job, *jobSize* $\in$ {1,2,3,4,5}. The *jobSize* value serves as a good indicator of the magnitude of the disturbance of the current schedule, which makes it an interesting parameter to investigate.

The initial schedule, for each month, was obtained using the genetic algorithm described in [33] with the objective of maximising the *Performance* measure. Each strategy S1-S8, as well as total rescheduling, right shift rescheduling and insertion in the end are used for rescheduling. In total rescheduling, all operations after *startPoint* are rescheduled with the algorithm from [33]. In right shift rescheduling, the operations after *startPoint* are shifted to the right as required in order to accommodate the new job. In insertion on the end, the new job is inserted after the completion of the last operation of the schedule. The *Performance* and *Stability* measures were recorded for each strategy on each instance.

All rescheduling strategies were implemented in Visual C++. Testing was performed on a Centrino Duo PC with 2.16 GHz, 1 GB of RAM and running Windows XP. Results are presented and analysed in the following section.

Table 1 Average *Performance* and *Stability* values obtained by the rescheduling strategies (larger vales are preferred)

		<i>Performance</i>										
		S1	S2	S3	S4	S5	S6	S7	S8	T	RS	E
<i>sat</i>	low	0.534	0.534	0.534	0.534	0.528	0.528	0.531	0.531	0.538	0.528	0.510
	medium	0.519	0.521	0.526	0.526	0.521	0.521	0.519	0.519	0.523	0.523	0.481
	high	0.528	0.528	0.528	0.529	0.496	0.505	0.500	0.498	0.518	0.513	0.519
<i>jobSize</i>	1	0.534	0.534	0.534	0.534	0.523	0.524	0.523	0.523	0.530	0.532	0.526
	2	0.532	0.532	0.532	0.532	0.518	0.521	0.519	0.519	0.523	0.529	0.516
	3	0.529	0.529	0.530	0.530	0.516	0.519	0.518	0.517	0.536	0.524	0.504
	4	0.520	0.526	0.526	0.526	0.513	0.517	0.514	0.513	0.522	0.515	0.491
	5	0.519	0.517	0.525	0.525	0.506	0.510	0.509	0.508	0.521	0.507	0.479
<i>insTime</i>	beginning	0.526	0.529	0.534	0.534	0.505	0.514	0.507	0.504	0.517	0.518	0.493
	middle	0.527	0.527	0.528	0.527	0.514	0.515	0.517	0.517	0.537	0.521	0.493
	end	0.526	0.526	0.526	0.527	0.526	0.526	0.526	0.526	0.526	0.526	0.525
total average		0.527	0.528	0.529	0.530	0.515	0.518	0.517	0.516	0.526	0.521	0.503
		<i>Stability</i>										
		S1	S2	S3	S4	S5	S6	S7	S8	T	RS	E
<i>sat</i>	low	0.982	0.983	0.977	0.977	0.982	0.981	0.982	0.979	0.912	1.000	1.000
	medium	0.965	0.965	0.966	0.966	0.966	0.967	0.966	0.965	0.943	1.000	1.000
	high	0.991	0.991	0.990	0.993	0.961	0.954	0.958	0.958	0.878	1.000	1.000
<i>jobSize</i>	1	0.998	0.998	0.998	0.998	0.978	0.978	0.977	0.977	0.894	1.000	1.000
	2	0.998	0.998	0.998	0.998	0.982	0.978	0.983	0.981	0.939	1.000	1.000
	3	0.990	0.990	0.989	0.989	0.986	0.982	0.987	0.987	0.919	1.000	1.000
	4	0.970	0.971	0.973	0.972	0.967	0.961	0.962	0.962	0.911	1.000	1.000
	5	0.941	0.942	0.932	0.936	0.935	0.938	0.935	0.930	0.892	1.000	1.000
<i>insTime</i>	beginning	0.973	0.975	0.972	0.973	0.947	0.940	0.944	0.942	0.838	1.000	1.000
	middle	0.982	0.982	0.977	0.980	0.979	0.979	0.979	0.977	0.918	1.000	1.000
	end	0.984	0.983	0.984	0.983	0.983	0.983	0.983	0.983	0.977	1.000	1.000
total average		0.979	0.980	0.978	0.979	0.970	0.967	0.969	0.967	0.911	1.000	1.000

4.1 Analysis and results

This section presents the results obtained by the different rescheduling strategies as well as the results of statistical analysis of the effects of the problem parameters and the match-up strategies on *Performance* and *Stability*. The results obtained by all strategies are summarised in Table 1, which presents the mean *Performance* and *Stability* attained by the investigated strategies for the instances grouped according to the different problem parameter values. We notice that, in general, the match-up strategies (S1–S8) are superior to total rescheduling (T), right shift (RS) and insertion in the end (E) with respect to *Performance* and superior to total rescheduling with respect to *Stability*. Among the match-up strategies, strategies S1–S4 seem to outperform strategies S5–S8 in most groups of instances. These results will be statistically verified next.

The statistical significance of the effects of problem parameters, match-up strategies, and the interactions among them on *Performance* and *Stability* was investigated by means of the Analysis of Variance (ANOVA). The summary of the ANOVA is given in Table 2, where the individual effects of problem parameters and rescheduling strategy on *Performance* and *Stability* are labelled “main effects”, whereas the combined effects of the pairs of variables are labelled “interactions”. The $A * B$ notation refers to the interaction between parameters A and B . Values under the heading F value and P value are the value of the Fisher statistic of the corresponding row effect, and the probability of this value being due to mere chance, respectively. Effects with a P value ≤ 0.05 are considered to be significant. The relatively large R^2 values for both metrics, *Performance* and *Stability*, is an indicator of the high power of the statistical

Table 2 Results of the ANOVA test for *Performance* and *Stability*

		<i>Performance</i>		<i>Stability</i>	
		<i>F</i> value	<i>P</i> value	<i>F</i> value	<i>P</i> value
Main effects					
	Strategy	13.56	≤ 0.01	55.38	≤ 0.01
	<i>sat</i>	104.94	≤ 0.01	7.92	≤ 0.01
	<i>jobSize</i>	25.47	≤ 0.01	81.17	≤ 0.01
	<i>insTime</i>	18.24	≤ 0.01	108.40	≤ 0.01
Interactions					
	Strategy* <i>sat</i>	6.49	≤ 0.01	8.06	≤ 0.01
	Strategy* <i>jobSize</i>	0.66	0.94	2.15	≤ 0.01
	Strategy* <i>insTime</i>	5.86	≤ 0.01	14.83	≤ 0.01
	<i>sat</i> * <i>jobSize</i>	1.93	0.05	13.25	≤ 0.01
	<i>sat</i> * <i>insTime</i>	30.01	≤ 0.01	18.74	≤ 0.01
	<i>jobSize</i> * <i>insTime</i>	0.43	0.90	1.95	0.05
<i>R</i> ²		0.71		0.83	

model implicit in the ANOVA test at explaining the observed variability in *Performance* and *Stability* [34].

The fact that the effect on variability due to *Strategy* is significant implies that some rescheduling strategies are better than the others. A pairwise comparison test using Bonferroni's correction [34] was carried out in order to identify which are those strategies that deliver higher *Performance* and *Stability* values. The results are given in Fig. 4, with comparisons of *Performance* below the diagonal, and above of *Stability*. Each field in Fig. 4 corresponds to the statistical test of the difference in means between the corresponding row-column strategies. The conclusion of each test is indicated with one of the following symbols: “←” which indicates that the strategy in the row is superior; “↑” which indicates that the column strategy is superior; and “=” which indicates that the strategies are non-distinguishable from each other. Additionally, the *t* statistic value and the *p* value are given on the top right and bottom right of each field, respectively. Large absolute *t* values and low *p* values confirm that the means of the corresponding row-column strategies are different, implying that one of them is superior. For example, the test in row S3, column RS, indicates that S3 obtains, on average, higher *Stability* values than RS. This holds with a *t*-test value of 5.70 and a $p \leq 0.05$. Not surprisingly, RS and E deliver good values for *Stability* and T for *Performance*. However, this is achieved at the price of poor *Stability* for T and poor *Performance* for RS and E. Remarkably, the results obtained by the match-up strategies are statistically non-distinguishable from T for *Performance* and from RS and E for *Stability*. This result indicates that the newly introduced approaches possess the best attributes of the investigated rescheduling approaches, but do not exhibit their weaknesses.

Figure 5(a) and 5(b) show these results graphically for *Performance* and *Stability*, respectively, using 95% confidence interval plots. In each plot, the dot indicates the average value on the whole set of instances obtained by the corresponding strategy in the *x*-axis. The vertical lines denote the 95% confidence interval of the mean value. Statistical differences are immediately detected when there is no overlap between the confidence intervals of two or more strategies. We notice that according to the interval plots in Fig. 5(a) and Fig. 5(b), strategies S1–S4 are the most competent with respect to both, *Performance* and *Stability*.

Strategies S5–S8 obtain, overall, low *Performance* values. In order to explain this behaviour, we have first measured the correlation between the *Performance* and the *Delay* that occur after rescheduling. The *Delay* represents the length of the overlap time period of the new schedule with the initial one after the rescheduling horizon. This is a consequence of rescheduling jobs within the rescheduling horizon after which some jobs are tardy (i.e. they do not meet their due dates). We measured the correlation between the *Performance* and the *Delay*, $\rho = -0.42$, indicating that high delays caused by the rescheduling process are associated with low *Performance* values. This explains the low *Performance* values achieved by strategies S5–S8 which, as shown in Fig. 5(c), are the ones that lead to the largest *Delay* values. On the other hand, S3 and S4, which seem to be the best performers according to Fig. 5(a), cause the smallest delays. Note that strategy E obtained low *Performance* values even when it produces no *Delay* value. This situation occurs due to a higher probability of the new jobs being tardy when they are inserted in the end of the schedule. We also notice, that the match-up algorithms are relatively efficient since they took 10 seconds on average in the rescheduling process (see Fig. 5(d)). This time is higher than the one required by RS and E, but lower than the one required by the total rescheduling. In any case, 10 seconds on average for the rescheduling of a one month production schedule is fast enough for a printing company industry and certainly for many other real world production shops.

Regarding the problem parameters, we recall from Table 2 that *sat*, *jobSize* and *insTime* have a significant influence on *Performance* and *Stability*. The nature of these effects is illustrated with the 95% confidence interval plots in Fig. 6. The *x*-axis of plots (a) and (d), (b) and (e), and (c) and (f), measures the level of *sat*, *jobSize* and *insTime*, respectively. The *y*-axis shows the average values of *Performance*, (a)–(c), and *Stability*, (d)–(f) over all rescheduling strategies. We observe that, in general, *Performance* and *Stability* values decrease when rescheduling is done on highly saturated schedules, rescheduling occurs at the beginning of the schedule, or when the arriving job requires many operations.

We also observe in Table 2 that all interactions of parameters are significant. Particularly interesting are those interactions involving *Strategy* and any of the problem parameters.

		Stability										
		S1	S2	S3	S4	S5	S6	S7	S8	T	RS	E
Performance	S1		= 0.14 1.00	= -0.39 1.00	= -0.19 1.00	= -2.52 0.65	= -3.07 0.12	= -2.74 0.33	= -3.09 0.11	← -17.51 ≤0.05	↑ 5.30 ≤0.05	↑ 5.30 ≤0.05
	S2	= 0.46 1.00		= -0.53 1.00	= -0.33 1.00	= -2.66 0.43	= -3.21 0.07	= -2.88 0.21	= -3.23 0.06	← -17.65 ≤0.05	↑ 5.16 ≤0.05	↑ 5.16 ≤0.05
	S3	= 1.40 1.00	= 0.94 1.00		= 0.20 1.00	= -2.13 1.00	= -2.67 0.41	= -2.35 1.00	= -2.69 0.39	← -17.12 ≤0.05	↑ 5.70 ≤0.05	↑ 5.70 ≤0.05
	S4	= 1.46 1.00	= 1.00 1.00	= 0.06 1.00		= -2.33 1.00	= -2.87 0.22	= -2.55 0.59	= -2.89 0.21	← -17.32 ≤0.05	↑ 5.50 ≤0.05	↑ 5.50 ≤0.05
	S5	↑ -5.99 ≤0.05	↑ -6.45 ≤0.05	↑ -7.39 ≤0.05	↑ -7.45 ≤0.05		= -0.55 1.00	= -0.22 1.00	= -0.57 1.00	← -14.99 ≤0.05	↑ 7.82 ≤0.05	↑ 7.82 ≤0.05
	S6	↑ -4.41 ≤0.05	↑ -4.87 ≤0.05	↑ -5.82 ≤0.05	↑ -5.87 ≤0.05	= 1.57 1.00		= 0.32 1.00	= -0.02 1.00	← -14.45 ≤0.05	↑ 8.37 ≤0.05	↑ 8.37 ≤0.05
	S7	↑ -5.14 ≤0.05	↑ -5.61 ≤0.05	↑ -6.55 ≤0.05	↑ -6.60 ≤0.05	= 0.84 1.00	= -0.73 1.00		= -0.34 1.00	← -14.77 ≤0.05	↑ 8.05 ≤0.05	↑ 8.05 ≤0.05
	S8	↑ -5.56 ≤0.05	↑ -6.02 ≤0.05	↑ -6.97 ≤0.05	↑ -7.02 ≤0.05	= 0.42 1.00	= -1.15 1.00	= -0.41 1.00		← -14.43 ≤0.05	↑ 8.39 ≤0.05	↑ 8.39 ≤0.05
	T	= -0.15 1.00	= -0.61 1.00	= -1.55 1.00	= -1.61 1.00	← 5.84 ≤0.05	← 4.26 ≤0.05	← 4.99 ≤0.05	← 5.41 ≤0.05		↑ 22.82 ≤0.05	↑ 22.82 ≤0.05
	RS	↑ -2.67 ≤0.05	↑ -3.13 ≤0.05	↑ -4.08 ≤0.05	↑ -4.13 ≤0.05	= 3.31 1.00	= 1.73 1.00	= 2.47 0.74	= 2.89 0.21	= -2.52 0.64		= 0.00 1.00
	E	↑ -12.07 ≤0.05	↑ -12.53 ≤0.05	↑ -13.47 ≤0.05	↑ -13.53 ≤0.05	↑ -6.07 ≤0.05	↑ -7.65 ≤0.05	↑ -6.92 ≤0.05	↑ -6.50 ≤0.05	↑ -11.92 ≤0.05	↑ -9.39 ≤0.05	

Fig. 4 Mean pairwise comparisons of *Performance* and *Stability*

These type of interactions indicate that some strategies are better at coping with certain problem conditions than others. That this is the case can be verified by Table 1. We have analysed the three interactions involving *Strategy* and have observed that strategies S1–S4 are either similar or superior to strategies S5–S8 under any scenario. This seems to contradict the existence of any interaction. However, the interactions exist because strategies S1–S4 are, under certain conditions, remarkably better than strategies S5–S8; under any other conditions they are only similar or slightly better.

Regarding the *insTime*, for instance, S1–S4 are much preferred if the arriving job is to be inserted at the beginning of the schedule. Under this scenario, the *Delay* inserted by the rescheduling algorithm will have a stronger effect than if rescheduling occurs at the end of the schedule, and consequently strategies S1–S4 will be remarkably superior to S5–S8. A similar reasoning explains why strategies S1–S4 are

also superior to S5–S8 when rescheduling occurs on highly saturated schedules (with high *sat* values). We have also observed that strategies S1–S4 are superior to S5–S8 when the arriving jobs have a small number of operations, 1 or 2. The number of operations to be inserted in the current schedule is directly proportional to the length of the rescheduling horizon. If the rescheduling horizon is relatively short, for example when inserting only one or two operations, the calculation of the *endPoint* performed differently by S1–S8 has a higher impact on the definition of the rescheduling horizon than if the rescheduling horizon is already large due to the insertion of a large number of operations. In this way, the small delay values obtained by S1–S4 lead to a better overall *Performance* and *Stability*.

Given our results in Table 1 and our statistical analysis, we can safely conclude that the match-up strategies are comparable to the right shift and insertion in the end strategies

Fig. 5 Overall results obtained by each rescheduling strategy; the x-axis shows the strategy; the y-axis shows the mean (dot) and 95% confidence interval (vertical bars) of *Performance* (a), *Stability* (b), *Delay* (c) and *Running Time* (d)

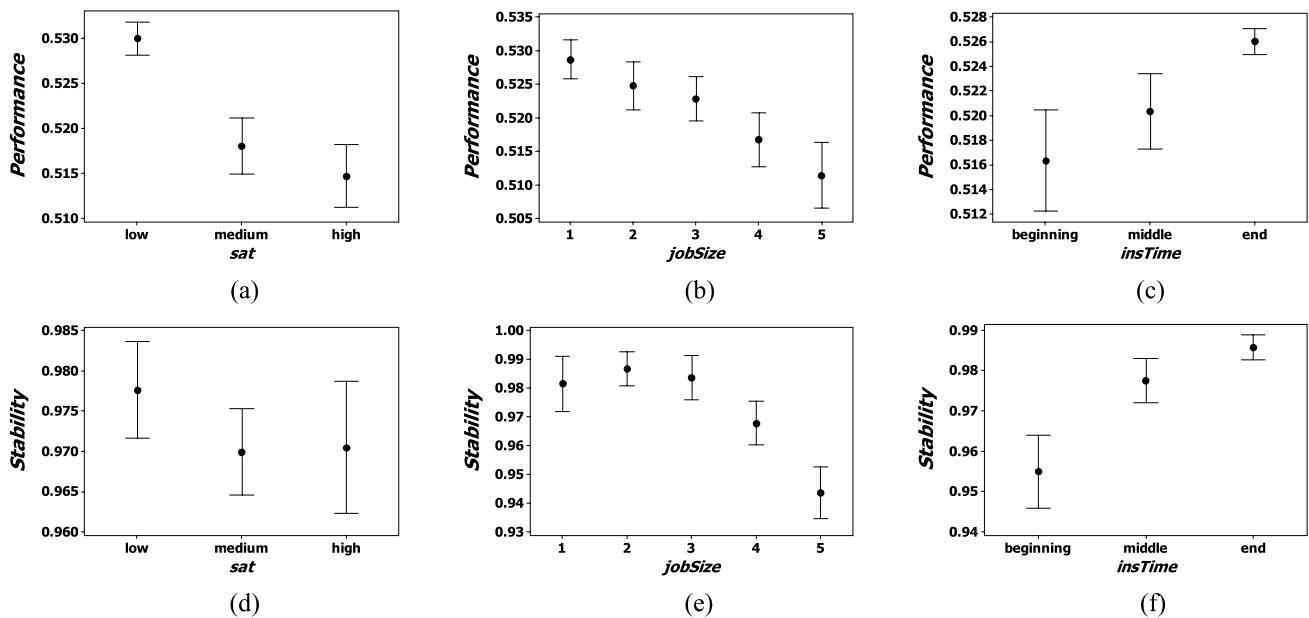
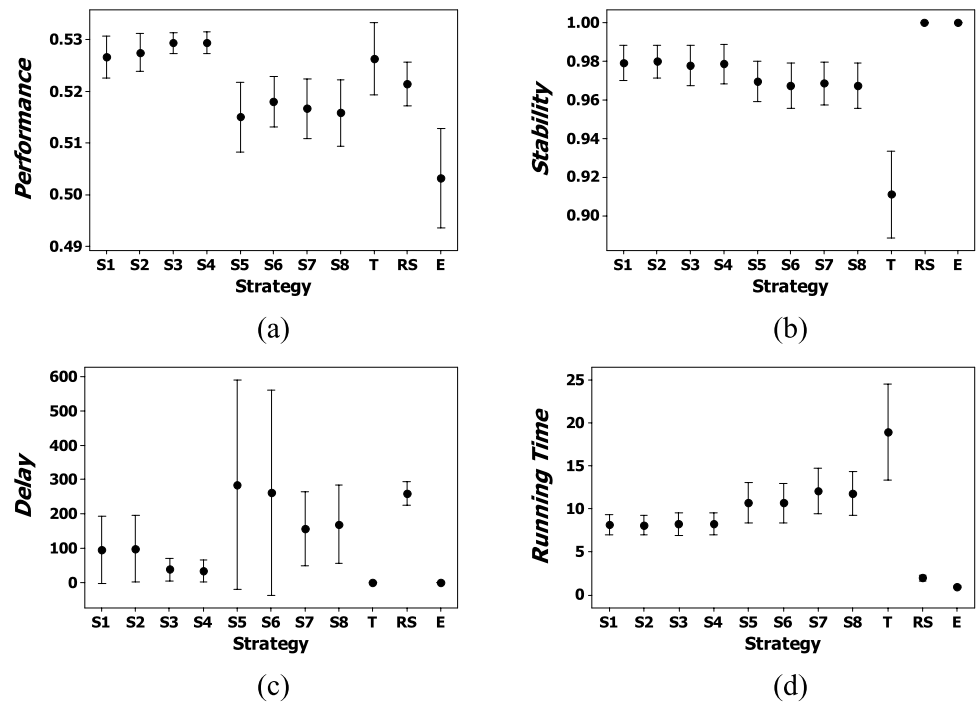


Fig. 6 Main effects on *Performance* (a), (b), (c) and *Stability* (d), (e), (f) due to *sat* (a), (d), *jobSize* (b), (e) and *insTime* (c), (f)

with respect to *Stability* and as good as the total rescheduling strategy with respect to *Performance*. The strategies S1–S4 seem to be slightly superior to the other match-up approaches and deliver the most consistent results under different problem scenarios, as demonstrated by Fig. 5. The strategies S3 and S4 can also be highlighted for possible incorporation into the scheduling system of Sherwood Press, since they produce high values with respect to *Performance*

and *Stability* and small *Delay* and *RunningTime* values, as shown in Fig. 5.

5 Conclusion and future work

This paper investigates a real world job shop rescheduling problem from a printing company. This problem is dynamic

in its nature since new orders arrive every day and need to be integrated in the current schedule. A match-up algorithm, which accommodates new orders by using available idle times on machines, is proposed. The motivation of the match up algorithms is to modify only a part of the initial schedule in such a way that the *Stability* and *Performance* of the shop floor are kept. Several strategies to define rescheduling horizons were proposed and compared with other strategies, including total rescheduling, right shift and insertion in the end of the schedule.

The obtained results were analysed and statistically validated. In summary, all match-up strategies do obtain reasonable values for both *Performance* and *Stability* regardless of the problem parameters. Strategies S1–S4 deliver better results because they set smaller rescheduling horizons than S5–S8. We have also observed that all match-up strategies are statistically non-distinguishable from total rescheduling with respect to *Performance* and comparable to the right shift and “insertion in the end” with respect to *Stability*. These encouraging results coupled with the fact that the proposed algorithms only take 10 seconds on average to reschedule a one month schedule, indicate that the proposed match-up strategies and particularly S1–S4 are adequate for the investigated and other similar production shops.

Further work considers other types of disruptions such as order changes, non-rush job arrivals, cancellation of jobs and rework required due to faults in product quality. In addition, resource disruptions such as machine breakdowns and sickness of workers will be also considered.

Acknowledgements The authors would like to acknowledge the support from the Engineering and Physics Science Research Council, UK (grant number GR/R95319/01) and Sherwood Press Ltd, Nottingham, UK.

References

1. Akkiraju R, Keskinocak P, Murthy S, Wu F (2001) An agent-based approach for scheduling multiple machines. *Appl Intell* 14:135–144
2. Leon V, Wu S, Storer R (1994) Robustness measures and robust scheduling for job shops. *IIE Trans* 26(5):32–43
3. Mason S, Jin S, Wessels C (2004) Rescheduling strategies for minimizing total weighted tardiness in complex job shops. *Int J Prod Res* 42(3):613–628
4. Pfeiffer A, Kádár B, Monostori L (2006) Stability-oriented evaluation of hybrid rescheduling methods in a job-shop with machine breakdowns. *CIRP J Manuf Syst* 35(6):563–570
5. Sun J, Xue D (2001) A dynamic reactive scheduling mechanism for responding to changes of production orders and manufacturing resources. *Comput Ind* 46(2):189–207
6. Cowling P, Johansson M (2002) Using real time information for effective dynamic scheduling. *Eur J Oper Res* 139(2):230–244
7. Sadeh N (1994) Micro-opportunistic scheduling: The micro-boss factory scheduler. In: *Intelligent scheduling*, 4. Morgan Kaufmann, San Mateo, pp 99–136
8. Smith S (1995) Reactive scheduling systems. In: Brown D, Scherer W (eds) *Intelligent scheduling systems*. Kluwer, Dordrecht
9. Rangsaritratamee R, Ferrell Jr W, Kurz M (2004) Dynamic rescheduling that simultaneously considers efficiency and stability. *Comput Ind Eng* 46(1):1–15
10. Bierwirth C, Mattfeld D (1999) Production scheduling and rescheduling with genetic algorithms. *Evol Comput* 7(1):1–17
11. Laguna M, Barnes J, Glover F (1993) Intelligent scheduling with tabu search: an application to jobs with linear delay penalties and sequence-dependent setup costs and times. *Appl Intell* 3:159–172
12. Filip F, Neagu G, Donciulescu D (1983) Job shop scheduling optimization in real-time production control. *Comput Ind* 4:395–403
13. Petrovic S, Petrovic D, Burke E (2009) Fuzzy logic based production scheduling/rescheduling in the presence of uncertainty. *Springer International Series in Operation Research and Management Science, Advancing the State-of-the-Art Subseries*
14. Vieira G, Herrmann J, Lin E (2003) Rescheduling manufacturing systems: a framework of strategies, policies and methods. *J Sched* 6(1):39–62
15. Aytug H, Lawley M, McKay K, Mohan S, Uzsoy R (2005) Executing production schedules in the face of uncertainties: a review and some future directions. *Eur J Oper Res* 161(1):86–110
16. Donath M, Graves R (1988) Flexible assembly systems: an approach for near real-time scheduling and routing of multiple products. *Int J Prod Res* 26(12):1903–1919
17. Chase C, Ramadge P (1992) On real-time scheduling policies for flexible manufacturing systems. *IEEE Trans Autom Control* 37(4):491–496
18. Gao H (1995) Building robust schedules using temporal protection an empirical study of constraint based scheduling under machine failure uncertainty. PhD thesis, Toronto University, Canada
19. Davenport A, Gefflot C, Beck J (2001) Slack-based techniques for robust schedules. In: *Proceedings of the sixth European conference on planning*, pp 7–18
20. Jensen M (2003) Generating robust and flexible job shop schedules using genetic algorithms. *IEEE Trans Evol Comput* 7(3):275–288
21. Arroyo-Figueroa G, Sucar L (2005) Temporal bayesian network of events for diagnosis and prediction in dynamic domains. *Appl Intell* 23:77–86
22. Li H, Li Z, Li L, Hu B (2000) A production rescheduling expert simulation system. *Eur J Oper Res* 124(2):283–293
23. Liao C, Chen W (2004) Scheduling under machine breakdown in a continuous process industry. *Comput Oper Res* 31(3):415–428
24. Duron C, Proth J, Wardi Y (2005) Insertion of a random task in a schedule: a real-time approach. *Eur J Oper Res* 164(1):52–63
25. Bean J, Birge J (1986) Match-up real-time scheduling. In: *Proceedings of the symposium on real-time optimization in automated manufacturing facilities*, national bureau of standards, special publication 724, pp 197–212
26. Birge J, Dempster M (1992) Optimality conditions for match-up strategies in stochastic scheduling. Technical Report 92-58, Department of Industrial and Operations Engineering, The University of Michigan
27. Birge J, Dempster M (1995) Optimal match-up strategies in stochastic scheduling. *Discrete Appl Math* 57(2–3):105–120
28. Bean J, Birge J, Mittenthal J, Noon C (1991) Matchup scheduling with multiple resources, release dates and disruptions. *Oper Res* 39(3):470–483
29. Akturk M, Gorgulu E (1999) Match-up scheduling under a machine breakdown. *Eur J Oper Res* 112(1):81–97
30. Smith S, Muscettola N, Matthys D, Ow P, Potvin J (1990) Opis: an opportunistic factory scheduling system. In: *IEA/AIE '90: Proceedings of the 3rd international conference on industrial and en-*

gineering applications of artificial intelligence and expert systems. ACM, New York, pp 268–274

31. Abumaizar R, Svestka J (1997) Rescheduling job shops under random disruptions. *Int J Prod Res* 35(7):2065–2082
32. Wu S, Storer R, Chang P (1993) One-machine rescheduling heuristics with efficiency and stability as criteria. *Comput Oper Res* 20(1):1–14
33. Petrovic S, Fayad C, Petrovic D, Burke E, Kendall G (2008) Fuzzy job shop scheduling with lot-sizing. *Ann Oper Res* 159(1):275–292
34. Miller R (1991) *Simultaneous statistical inference*. Springer, New York



Patrick Moratori was an undergraduate at the Federal University of Juiz de Fora and then a M.Sc. student at the Federal University of Rio de Janeiro, working in the field of Computational Intelligence, Evolutionary Algorithms and Computational Games. Today he is a Ph.D. student at the University of Nottingham and his research interests lie in the fields of Production Scheduling, Operational Research and Fuzzy systems.



Sanja Petrovic has conducted multidisciplinary research into models, heuristics and algorithms for a variety of real world scheduling problems, including radiotherapy planning and scheduling, production scheduling, nurse rostering, and university timetabling. Her research interest includes the fields of case-based reasoning, multicriteria decision analysis, modelling of uncertainty by fuzzy sets, meta-heuristics, hyper-heuristics and multi-agent systems.



José Antonio Vázquez-Rodríguez obtained his B.Sc. (Industrial and Systems Engineering) from Tecnológico de Monterrey, Mexico. After graduation he worked for two years as a production manager at Plega Envases de Occidente S.A. de C.V., PEOSA for short, which is a cardboard box factory based in Mexico. He obtained a research grant from CONACYT and came to study a Ph.D. in Operational Research and Computing Science at the Mathematical Sciences Department, University of Essex, U.K. His research topic involved the design of heuristic methods to solve scheduling problems. Today he is a research fellow at the ASAP group, University of Nottingham.